



GCM TICKETS

Sistema de Soporte Técnico

Documentación Técnica y de Funcionamiento — Arquitectura API + Web + Base de Datos

Versión	3.0.0
---------	-------

Fecha	15 de julio de 2026
-------	---------------------

Servidor	Windows Server 2022 Datacenter
----------	--------------------------------

IP del Servidor	10.0.1.108
-----------------	------------

Puertos de servicio	3000 (API) / 3001 (Web)
---------------------	-------------------------

Stack	Node.js + Express + TypeScript / Next.js + React + Tailwind / SQL Server
-------	--

Contenido

1.	Resumen Ejecutivo	3
2.	Arquitectura General	3
3.	Stack Tecnológico Completo	4
4.	Estructura del Proyecto	6
5.	Base de Datos	7
6.	Referencia de la API REST	9
7.	Autenticación y Autorización	12
8.	Seguridad	13
9.	Funcionalidades del Sistema	16
10.	Roles y Permisos de Usuario	18
11.	Gestión de Archivos y Subida Móvil	18
12.	Despliegue y Configuración del Servidor	19
13.	Respaldo y Recuperación	21
14.	Mantenimiento y Comandos	21
15.	Historial de Migración	22

1. Resumen Ejecutivo

GCM Tickets es el sistema interno de gestión de tickets de soporte técnico de Grupo Milcien S.A. de C.V. Permite a los empleados reportar incidencias, solicitar soporte y dar seguimiento a sus casos, mientras el equipo de TI gestiona, asigna y resuelve las solicitudes desde un panel de administración centralizado. El sistema también controla el inventario de equipos, los préstamos a empleados y mantiene una bitácora de auditoría de toda acción administrativa relevante.

Desde julio de 2026 el sistema opera bajo una arquitectura de **dos servicios independientes** que antes eran uno solo: una **API backend** (Express + TypeScript) que concentra toda la lógica de negocio y el acceso a la base de datos, y un **frontend web** (Next.js + TypeScript + Tailwind) que reemplazó al sitio HTML/JavaScript plano original. Ambos siguen ejecutándose sobre la misma máquina Windows Server, pero ahora como dos procesos Node.js separados que se comunican por HTTP/JSON.

Esta versión de la documentación (3.0.0) reemplaza a la 2.0.0 (30 de junio de 2026) para reflejar la migración completa del frontend y el backend a TypeScript, la nueva arquitectura de dos puertos, y una revisión y endurecimiento de seguridad realizada sobre todo el sistema (sección 8).

2. Arquitectura General

El sistema sigue un modelo cliente-servidor de tres capas, ahora con el cliente y el servidor de aplicación como procesos completamente separados:



2.1 Por qué dos procesos separados

- Separación de responsabilidades:** el backend expone únicamente una API JSON (`/api/*`) y sirve los archivos adjuntos (`/uploads`); no conoce nada de HTML/React. El frontend Next.js no toca la base de datos directamente ni contiene lógica de negocio — solo consume la API.
- Sin sesión de servidor:** la autenticación es 100% por **Bearer token (JWT)** en el header `Authorization` , nunca por cookies. Esto es lo que permite que ambos procesos vivan en puertos (orígenes) distintos sin necesitar un mecanismo de sesión compartida.
- CORS abierto por diseño:** el backend responde con `Access-Control-Allow-Origin` reflejando cualquier origen (`cors({''} origin: true {''})`). Esto es intencional y seguro en este caso concreto: como no hay cookies de sesión, un sitio externo no tiene forma de obtener el token de otro usuario para adjuntarlo a sus propias peticiones. Se necesita porque la app se usa desde cualquier dispositivo de la red local por IP (por ejemplo, un celular escaneando el código QR de subida de evidencia no tiene un origen fijo conocido de antemano).

2.2 Comunicación entre capas

De	A	Protocolo	Detalle
Navegador (admin/portal)	web/ (Next.js)	HTTP	Renderizado de páginas React (App Router), assets estáticos
web/ (cliente, en el navegador)	backend/ (Express)	HTTP + JSON	Todas las operaciones de datos vía <code>fetch</code> , token JWT en cada request
backend/	SQL Server	TDS (mssql)	Queries parametrizadas y stored procedures

De	A	Protocolo	Detalle
Celular (QR)	web/ → backend/	HTTP + multipart	Sube el archivo sin necesidad de iniciar sesión en el teléfono (token de sesión de subida temporal)

3. Stack Tecnológico Completo

3.1 Backend — backend/

Paquete	Versión	Qué hace en este proyecto
express	^4.18.2	Framework HTTP — enruta y sirve todos los endpoints <code>/api/*</code> y los archivos de <code>/uploads</code> .
typescript	^7.0.2	Todo el backend está escrito en TypeScript (<code>.ts</code>) y se compila a <code>dist/</code> antes de ejecutarse.
tsx	^4.23.1	Ejecuta TypeScript directamente en desarrollo (<code>npm run dev</code>) con recarga automática, sin compilar a mano.
mssql	^12.5.5	Driver oficial de SQL Server para Node.js. Todas las queries usan parámetros con <code>.input()</code> , nunca concatenación de texto — esto es lo que evita inyección SQL.
jsonwebtoken	^9.0.3	Firma y verifica los tokens JWT de sesión (algoritmo HS256, 12 horas de validez).
bcrypt	^6.0.0	Hashea las contraseñas de usuario (12 rondas de costo). Nunca se guarda una contraseña en texto plano.
cors	^2.8.6	Middleware de CORS — habilita que el frontend (otro puerto/origen) pueda llamar a la API.
helmet	^8.3.0	(Nuevo) Aplica cabeceras HTTP de seguridad por defecto (X-Content-Type-Options, X-Frame-Options, Referrer-Policy, HSTS, etc.) a cada respuesta.
express-rate-limit	^8.5.2	(Nuevo) Limita intentos de <code>/auth/login</code> (10 cada 15 min por IP) y <code>/auth/register</code> (10 cada hora), como protección contra fuerza bruta.
multer	^2.1.1	Procesa la subida de archivos adjuntos (multipart/form-data): valida tipo, tamaño y genera un nombre de archivo aleatorio en disco.
qrcode	^1.5.4	Genera la imagen del código QR que el empleado escanea desde su celular para subir evidencia a un ticket.
nodemailer	^9.0.3	Envía las notificaciones por correo (nuevo ticket, cambio de estado, nueva solicitud de registro) vía SMTP, opcional.
dotenv	^16.6.1	Carga las variables de <code>backend/.env</code> al proceso en tiempo de ejecución.

3.2 Frontend — web/

Paquete	Versión	Qué hace en este proyecto
next	16.2.10	Framework de React usado en modo App Router. Define las rutas por carpeta (<code>/admin</code> , <code>/portal</code> , <code>/registro</code> , etc.) y compila la app.
react / react-dom	19.2.4	Librería de UI — construye toda la interfaz como componentes.
typescript	^5	Igual que en el backend: todo el frontend es <code>.ts</code> / <code>.tsx</code> , sin JavaScript plano.
tailwindcss	^4	Sistema de estilos utilitario (CSS-first, vía <code>@theme</code>). Define los tokens de color de marca (paleta <i>admin</i> y paleta <i>portal</i>) y el modo oscuro.
swr	^2.4.2	Librería de fetching de datos — cachea y refresca automáticamente los datos de la API (tickets, inventario, notificaciones) sin escribir <code>setInterval</code> manuales.
chart.js / react-chartjs-2	^4.5.1 / ^5.3.1	Gráficos del dashboard de tickets (donas de estado/prioridad, barras de actividad).
xlsx	^0.18.5	Genera el archivo Excel de exportación de tickets desde el navegador (solo escritura — ver nota de seguridad en 8.6).

Paquete	Versión	Qué hace en este proyecto
@tabler/icons-react	^3.44.0	Set de iconos usado en toda la interfaz (reemplazó al font-icon por CDN de la versión anterior).
clsx	^2.1.1	Utilidad pequeña para combinar clases CSS condicionalmente.
eslint / eslint-config-next	^9 / 16.2.10	Linting — incluye las reglas nuevas de React 19 (<code>react-hooks/purity</code> , <code>react-hooks/set-state-in-effect</code>) que el código respeta.

Principio de la migración: "si ya usábamos la tecnología, no se cambia" — Express, JWT y SQL Server se mantuvieron intactos en su lógica; el trabajo fue tipar el backend a TypeScript y reconstruir el frontend (antes HTML + JS + CSS planos servidos como archivos estáticos) en Next.js/TypeScript/Tailwind, replicando el diseño visual existente sin rediseñarlo.

4. Estructura del Proyecto

El repositorio está dividido en dos proyectos Node.js completamente independientes, cada uno con su propio `package.json`, `node_modules` y ciclo de build:

```
SistemaAPP/
├─ start.bat                - Arranque del backend en Windows
├─ start-web.bat            - Arranque del frontend en Windows
├─ ecosystem.config.js      - Config de PM2 (backend + frontend) para producción
├─ setup-servidor.ps1       - Instalación como servicio permanente (Windows Server)
├─ backup.ps1               - Backup diario de BD + uploads
├─
├─ backend/                 - API (Express + TypeScript + JWT)
│   ├── server.ts           - Punto de entrada (CORS, helmet, rutas, /uploads)
│   ├── tsconfig.json
│   ├── package.json
│   ├── .env / .env.example - Configuración (BD, JWT, SMTP, puertos)
│   ├── setup.js            - Crea la BD, tablas, procedimientos y el admin inicial
│   ├── schema.sql          - Tablas, roles, seeds y migraciones
│   ├── procedimientos.sql   - Stored Procedures (19 en total)
│   ├── assets/gcm.jpg       - Logo usado en los correos de notificación
│   ├── uploads/            - Adjuntos subidos por los usuarios (no versionado)
│   └─ src/
│       ├── db.ts           - Pool de conexión, helpers query/exec, bcrypt
│       ├── config.ts        - Variables de entorno validadas (SESSION_SECRET, puertos)
│       ├── helpers.ts       - Auditoría, formato de fechas, URL del frontend
│       ├── ticketloader.ts  - Construye el DTO "Ticket" desde la BD
│       ├── mobileSessions.ts - Sesiones temporales de subida por QR (en memoria)
│       ├── mailer.ts        - Notificaciones por correo (nodemailer)
│       ├── types.ts         - Tipos TypeScript compartidos del backend
│       ├── middleware/
│       │   ├── auth.ts      - requireAuth / requireRole / requireSuperadminOrAcceso
│       │   └─ upload.ts     - Validación y almacenamiento de adjuntos (multer)
│       └─ routes/
│           ├── auth.ts      - Login y registro (con rate limiting)
│           ├── tickets.ts   - CRUD de tickets, comentarios, notas, adjuntos
│           ├── usuarios.ts  - Usuarios, solicitudes de acceso y permisos
│           ├── inventory.ts - Inventario, préstamos y bitácora
│           └─ mobileUpload.ts - Sesión y subida de adjuntos desde celular vía QR
├─
└─ web/                     - Frontend (Next.js + TypeScript + Tailwind)
    ├── app/
    │   ├── portal/         - Portal de empleados
    │   ├── admin/          - Panel TI / administración
    │   │   ├── inventario/ - Equipos, préstamos, responsables (dashboard)
    │   │   ├── usuarios/   - Gestión de usuarios y solicitudes de acceso
    │   │   └─ auditoria/   - Bitácora del sistema
    │   ├── registro/       - Solicitud de acceso (empleados nuevos)
    │   └─ mobile-upload/    - Subida de adjuntos desde el celular (QR)
    ├── components/ui/      - Modal, Toast, Tabs, Autocomplete, StatCard, etc.
    └─ lib/                 - Cliente API (fetch + Bearer), auth, tipos, tema oscuro
```

5. Base de Datos

Microsoft SQL Server, base de datos `gcm_tickets`. Toda la lógica de negocio pesada vive en **stored procedures** (`procedimientos.sql`); las queries simples/ad-hoc se ejecutan como SQL parametrizado inline desde `db.ts`. En ningún caso se concatena texto del usuario dentro de una query — ver sección 8.2.

5.1 Tablas principales

Tabla	Propósito
<code>roles</code>	Catálogo de 4 roles (empleado, tecnico, admin, superadmin) con su <code>nivel</code> numérico (1–4), usado por todo el sistema de permisos.
<code>usuarios</code>	Toda cuenta del sistema — empleados del portal y personal del panel admin viven en la misma tabla, diferenciados por <code>rol_id</code> . Incluye contraseña hasheada, departamento, columnas <code>acceso_*</code> (permisos por módulo) y estado de aprobación.
<code>departamentos</code>	Catálogo de departamentos de la empresa (Sistemas/IT, Administración, RRHH, etc.).
<code>solicitudes_registro</code>	Peticiones de alta de empleados nuevos, pendientes de aprobación por un administrador.
<code>tickets</code>	Incidencias reportadas. Guarda estado, prioridad, categoría, quién lo reportó (<code>reporter_id</code>) y a quién está asignado (<code>asignado_id</code>).
<code>comentarios</code>	Respuestas de un ticket. El flag <code>es_interno</code> distingue una respuesta visible al empleado de una nota interna solo para staff.
<code>historial_tickets</code>	Bitácora de cambios de cada ticket (quién, qué campo, valor anterior/nuevo, cuándo).
<code>adjuntos</code>	Archivos (imagen/video) asociados a un ticket — guarda solo la referencia; el archivo físico vive en <code>backend/uploads/</code> .
<code>dispositivos</code>	Equipos recibidos en taller para reparación (distinto del inventario general).
<code>inventario</code>	Equipos de TI de la empresa — laptops, routers, etc. Admite manejo "por unidad" (serie única) o "por cantidad" (lote sin serie individual).
<code>historial_inventario</code>	Bitácora de cambios sobre un ítem de inventario.
<code>prestamos</code>	Préstamo de un equipo/lote a un empleado — fecha de devolución estimada/real, autorizado por, y soporta devolución parcial en préstamos por cantidad.
<code>auditoria</code>	Bitácora general de acciones administrativas (crear/eliminar usuario, cambiar permisos, eliminar ticket, etc.), independiente del historial de un ticket o equipo específico.
<code>contadores</code>	Secuencias propias para generar IDs legibles con prefijo (<code>TK-001</code> , <code>INV-014</code> , <code>PREST-009</code>), incrementadas de forma atómica.

5.2 Relaciones clave

- `usuarios.rol_id` → `roles.id` — determina el nivel de acceso (1 a 4) de cada cuenta.
- `usuarios.departamento_id` → `departamentos.id` — usado para mostrar y filtrar por departamento.
- `tickets.reporter_id` / `tickets.asignado_id` → `usuarios.id` — quién reportó y quién atiende.
- `comentarios.ticket_id` , `historial_tickets.ticket_id` , `adjuntos.ticket_id` → `tickets.id` (ON DELETE CASCADE — se limpian solos si se borra el ticket).
- `prestamos.inventario_id` → `inventario.id` , `prestamos.empleado_id` → `usuarios.id`.

5.3 Stored procedures (19)

Grupo	Procedimientos
Autenticación y registro	sp_GetUserForLogin, sp_UpdateLastLogin, sp_InsertSolicitudRegistro, sp_GetSolicitudes, sp_AprobarSolicitud, sp_RechazarSolicitud
Usuarios	sp_GetUsuarios, sp_GetAdmins
Tickets	sp_GetTickets, sp_GetTicket, sp_GetComentarios, sp_GetHistorialTicket, sp_GetAdjuntos, sp_GetDevices, sp_CrearDispositivoConTicket
Inventario y préstamos	sp_GetInventory, sp_GetLoans, sp_GetDepartamentos
Auditoría	sp_LogAudit, sp_GetAuditoria

Las operaciones de escritura más simples (INSERT/UPDATE de un solo registro) se hacen como queries parametrizadas inline desde `db.ts` en vez de procedimientos, para no multiplicar stored procedures de una sola línea.

6. Referencia de la API REST

Todas las rutas cuelgan de `/api`. Salvo donde se indique **público**, cada endpoint requiere un header `Authorization: Bearer <token>` válido.

6.1 Auth — `/api/auth`

Método	Ruta	Auth	Descripción
POST	<code>/login</code>	Público (rate-limited: 10/15min)	Valida usuario/contraseña, devuelve JWT + datos de sesión y permisos por módulo.
POST	<code>/register</code>	Público (rate-limited: 10/hora)	Crea una solicitud de registro de empleado, pendiente de aprobación.

6.2 Tickets — `/api/tickets`

Método	Ruta	Auth	Descripción
GET	<code>/</code>	Autenticado	Lista tickets. Staff (nivel ≥ 2) ve todos; un empleado solo ve los suyos y nunca las notas internas.
POST	<code>/</code>	Autenticado	Crea un ticket nuevo (el reporter es siempre el usuario autenticado).
GET	<code>/:id</code>	Autenticado	Detalle de un ticket. 404 si un empleado intenta ver uno ajeno.
PATCH	<code>/:id</code>	Staff (nivel ≥ 2)	Cambia estado, prioridad, categoría, título, descripción o asignación.
DELETE	<code>/:id</code>	Staff (nivel ≥ 2)	Elimina el ticket y sus adjuntos en disco.
POST	<code>/:id/comments</code>	Dueño del ticket o staff	Agrega una respuesta visible para el empleado.
POST	<code>/:id/notes</code>	Staff (nivel ≥ 2)	Agrega una nota interna, nunca visible para el empleado.
POST	<code>/:id/attachments</code>	Dueño del ticket o staff	Sube un adjunto (imagen/video) desde el navegador.
POST	<code>/:id/attachments/from-mobile</code>	Dueño del ticket o staff	Asocia al ticket un archivo ya subido desde el celular vía QR.

6.3 Usuarios — `/api`

Método	Ruta	Auth	Descripción
GET	<code>/usuarios</code>	Superadmin o permiso "usuarios"	Lista de empleados registrados.
GET	<code>/usuarios/:id/tickets</code>	Superadmin o permiso "usuarios"	Tickets de un empleado específico.
DELETE	<code>/usuarios/:id</code>	Admin (nivel ≥ 3)	Elimina una cuenta y limpia sus referencias en préstamos/solicitudes.
PATCH	<code>/usuarios/:id/password</code>	Superadmin (nivel 4)	Cambia la contraseña de cualquier usuario.
PATCH	<code>/usuarios/:id/activo</code>	Admin (nivel ≥ 3)	Activa/desactiva una cuenta.
PATCH	<code>/usuarios/:id/permisos</code>	Superadmin (nivel 4)	Otorga/revoca acceso a los módulos restringidos.

Método	Ruta	Auth	Descripción
GET	/usuarios/lista	Staff (nivel ≥2)	Lista liviana (id, nombre, si es portal, departamento/rol) para autocompletados.
GET	/admins	Staff (nivel ≥2)	Lista del personal del panel admin (técnicos/admins/superadmin).
POST	/admins	Admin (nivel ≥3)	Crea una cuenta de staff.
DELETE	/admins/:username	Admin (nivel ≥3)	Elimina una cuenta de staff.
GET	/solicitudes	Superadmin o permiso "solicitudes"	Solicitudes de registro pendientes/revisadas.
POST	/solicitudes/:id/aprobar	Superadmin o permiso "solicitudes"	Aprueba y crea la cuenta del empleado.
POST	/solicitudes/:id/rechazar	Superadmin o permiso "solicitudes"	Rechaza la solicitud con un motivo.

6.4 Inventario, préstamos y bitácora — /api

Método	Ruta	Auth	Descripción
GET / POST	/devices	Superadmin o permiso "inventario"	Equipos recibidos en taller.
GET / POST	/inventory	Superadmin o permiso "inventario"	Alta y listado de inventario TI.
PATCH / DELETE	/inventory/:id	Superadmin o permiso "inventario"	Editar o eliminar un equipo.
GET / POST	/loans	Superadmin o permiso "prestamos"	Listar y registrar préstamos.
PATCH	/loans/:id	Superadmin o permiso "prestamos"	Registrar devolución total o parcial.
GET	/auditoria	Superadmin o permiso "bitacora"	Bitácora de acciones administrativas, filtrable.

6.5 Subida móvil — /api/mobile-upload

Método	Ruta	Auth	Descripción
POST	/session	Público (por diseño)	Crea una sesión de subida temporal (token aleatorio de 128 bits, expira en 5 min).
GET	/qr/:token	Público	Genera la imagen QR con la URL de subida para ese token.
POST	/:token	Público (protegido por el token)	El celular sube el archivo a esta sesión.
GET	/status/:token	Público	El navegador hace polling para saber si el celular ya subió el archivo.

Estos 4 endpoints son intencionalmente públicos: el celular que escanea el QR nunca inicia sesión. La seguridad viene de que el token es aleatorio (32 caracteres hex, no adivinable) y expira a los 5 minutos.

7. Autenticación y Autorización

7.1 Flujo de autenticación

1. El usuario envía `usuario/contraseña` a `POST /api/auth/login`.
2. El backend compara la contraseña con el hash bcrypt guardado en `usuarios.password_hash`.
3. Si es correcta, se firma un JWT (HS256, 12 horas) con el payload: `{'id', 'username', 'nombre', 'rol_nivel'}`.
4. El frontend guarda el token (y los datos de sesión) en `sessionStorage` del navegador — se pierde al cerrar la pestaña, no persiste entre sesiones del sistema operativo.
5. Cada request posterior a la API incluye `Authorization: Bearer <token>`. El middleware `requireAuth` lo verifica y adjunta `req.user` a la request.

7.2 Niveles de rol

Nivel	Rol	Alcance
1	Empleado	Portal — solo sus propios tickets, nunca notas internas.
2	Técnico	Panel admin — gestiona cualquier ticket (staff).
3	Admin	Panel admin — gestión completa de tickets, altas de técnicos, usuarios.
4	Superadmin	Todo lo anterior + control total, incluye otorgar permisos por módulo.

7.3 Middleware de autorización

Función	Uso
<code>requireAuth</code>	Exige un JWT válido. No mira el rol.
<code>requireRole(n)</code>	Exige JWT válido + <code>rol_nivel ≥ n</code> .
<code>requireSuperadminOrAcceso(modulo)</code>	El superadmin (nivel 4) siempre pasa. Cualquier otro usuario necesita que la columna <code>acceso_<modulo></code> de su fila en <code>usuarios</code> esté en <code>true</code> — se consulta la BD en cada request (no se confía en el JWT), así una revocación de permiso surte efecto de inmediato sin esperar a que el token expire.

7.4 Permisos por módulo

Cinco secciones del panel admin están restringidas por defecto a solo el superadmin, y se otorgan una por una desde "Usuarios administradores": `inventario`, `prestamos`, `bitacora`, `solicitudes`, `usuarios`. Cambiar la contraseña de otro usuario está reservado exclusivamente al superadmin sin excepción.

8. Seguridad

Esta sección documenta una auditoría de seguridad completa realizada sobre el sistema en julio de 2026, junto con las correcciones aplicadas. Se revisó: autenticación/JWT, inyección SQL, manejo de archivos, CORS, gestión de secretos, control de acceso (IDOR), XSS, manejo de errores y dependencias vulnerables.

8.1 Gestión de secretos

- `backend/.env` está excluido de git (`.gitignore`) y nunca se versiona.
- **Corrección:** se detectaron commits antiguos del repositorio que sí contenían el archivo `.env` real (con `SESSION_SECRET` , `DB_PASSWORD` y la contraseña de aplicación SMTP). Se reescribió el historial completo de git para eliminar ese contenido de todos los commits.
- **Corrección:** `db.ts` y `setup.js` tenían la contraseña de la base de datos de desarrollo como valor por defecto hardcodeado en el código fuente si `DB_PASSWORD` faltaba en `.env` . Ahora el proceso falla al arrancar si no está configurada explícitamente — igual que ya ocurría con `SESSION_SECRET` .
- **Corrección:** `config.ts` ahora exige que `SESSION_SECRET` tenga al menos 32 caracteres al arrancar, para evitar un secreto débil por descuido.

8.2 Inyección SQL

Revisado en `db.ts` y en las 5 rutas del backend: el 100% de las queries usa parámetros posicionales (`?` → `.input()`) o stored procedures con parámetros nombrados. Ningún dato de la request del cliente se concatena directamente en un texto SQL. Los nombres de columna/tabla dinámicos que sí aparecen en algunos `template literals` provienen siempre de un mapa fijo definido en el propio servidor (whitelist), nunca de un valor libre del cliente.

8.3 Control de acceso — corrección de IDOR en tickets

Hallazgo (corregido): antes de esta revisión, `GET /api/tickets` devolvía *todos* los tickets del sistema a cualquier usuario autenticado, y el portal de empleados solo filtraba la lista *del lado del cliente*. Cualquier empleado podía, llamando directamente a la API, ver, comentar, adjuntar archivos, editar o eliminar tickets ajenos, e incluso leer y escribir las notas internas de staff.

Corrección aplicada en `backend/src/routes/tickets.ts` :

- El listado y el detalle de tickets ahora filtran por `reporter_id` del lado del servidor cuando quien pregunta es un empleado (nivel 1); un ticket ajeno responde `404` (no `403` , para no confirmar siquiera que existe).
- Las notas internas (`notes`) se eliminan de la respuesta JSON para cualquier no-staff, incluso en sus propios tickets.
- `PATCH` , `DELETE` y "agregar nota interna" ahora exigen nivel de staff (≥ 2) — un empleado nunca pudo hacerlo desde la interfaz, pero antes sí podía vía API directa.
- Comentar y adjuntar archivos se permite al dueño del ticket o a cualquier miembro de staff, verificado contra `reporter_id` en la base de datos, no contra lo que el cliente diga.

8.4 Autenticación — endurecimiento

- **Rate limiting** (nuevo): `/auth/login` limitado a 10 intentos / 15 min por IP (no cuenta logins exitosos); `/auth/register` a 10 / hora — mitiga fuerza bruta de contraseña.
- **Algoritmo JWT fijado** (nuevo): `jwt.verify` ahora especifica explícitamente `algorithms: ['HS256']` en vez de aceptar cualquier algoritmo que traiga el token.
- Los mensajes de error de login son deliberadamente genéricos ("Usuario o contraseña incorrectos") tanto para usuario inexistente, contraseña incorrecta, o intento de entrar al portal que no corresponde al nivel de la cuenta — no se revela si un usuario existe.
- Contraseñas: bcrypt, 12 rondas de costo. Política mínima de 6 caracteres en registro y cambio de contraseña (sin requisito de complejidad adicional actualmente).

8.5 Cabeceras HTTP y CORS

- **helmet** (nuevo) agrega cabeceras de seguridad estándar a toda respuesta: `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: no-referrer`, `Strict-Transport-Security`, etc. La política de Content-Security-Policy se desactiva a propósito porque este servidor solo expone JSON y archivos estáticos de `/uploads`, nunca HTML propio; y `Cross-Origin-Resource-Policy` se relaja a `cross-origin` porque el frontend (otro puerto) necesita cargar imágenes/videos de `/uploads` directamente.
- **CORS** (`origin: true`): justificado en la sección 2.1 — seguro en este diseño concreto porque la autenticación es 100% por Bearer token, nunca por cookies.

8.6 Manejo de archivos subidos

Control	Estado
Tamaño máximo	50 MB por archivo, aplicado por multer
Tipos permitidos	Whitelist por extensión (imágenes/video); no se ejecuta ningún archivo subido
Nombre en disco	Aleatorio (<code>crypto.randomBytes</code>), nunca el nombre original — evita path traversal vía <code>../</code>
Sesión de subida móvil	Token de 128 bits, expira en 5 minutos — no es adivinable ni de fuerza bruta viable
Servido de <code>/uploads</code>	Estático, sin autenticación — mitigado por nombres de archivo aleatorios no listables; queda como mejora futura (ver 8.8)

8.7 Manejo de errores

Se corrigieron dos endpoints en `usuarios.ts` (eliminar usuario, cambiar contraseña) que devolvían el `message` crudo del error de base de datos al cliente. Ahora, como el resto del sistema, registran el detalle en el log del servidor y devuelven un mensaje genérico en español al cliente.

8.8 Dependencias — estado de `npm audit`

Backend

0 vulnerabilidades. Se actualizó `bcrypt` a 6.x, `nodemailer` a 9.x y `multer` a su última versión — esto eliminó 4 vulnerabilidades HIGH (incluyendo las que arrastraba `tar` vía la cadena de dependencias de compilación de `bcrypt`).

Frontend

1 vulnerabilidad HIGH sin parche disponible: `xlsx` (prototype pollution / ReDoS, disparado al *leer* un archivo malicioso). Evaluada como **no explotable** en este proyecto: la librería solo se usa para *generar* el Excel de exportación desde datos internos, nunca para leer archivos subidos por un usuario.

8.9 Pendientes / decisiones del usuario

- La contraseña de aplicación de Gmail (SMTP) que quedó expuesta en el historial de git antiguo debe rotarse manualmente desde la cuenta de Google (`myaccount.google.com/apppasswords`) — pendiente.
- `SESSION_SECRET` y `DB_PASSWORD` actuales corresponden al entorno de desarrollo; se decidió no rotarlos todavía. Deben generarse valores nuevos y de alta entropía antes de cualquier despliegue a producción real.
- El acceso a `/uploads` sigue sin requerir sesión (mitigado por nombres aleatorios, no por autenticación real) — quedaría como mejora futura si se requiere control de acceso estricto sobre adjuntos.

9. Funcionalidades del Sistema

9.1 Portal de empleados — `/portal`

- Registro con nombre, usuario, correo, teléfono, departamento, finca y área — la cuenta queda pendiente hasta aprobación de un administrador.
- Creación de tickets con prioridad, categoría y adjuntos.
- Subida de adjuntos desde el celular escaneando un código QR, sin iniciar sesión en el teléfono.
- Seguimiento del estado e historial de sus propios tickets, con respuesta a comentarios de soporte.

9.2 Panel de administración — `/admin`

- **Tickets:** bandeja con filtros, gráficos (Chart.js), asignación a técnicos, cambio de estado, comentarios y notas internas, exportación a Excel, alertas de SLA.
- **Usuarios** (`/admin/usuarios`): lista de empleados y staff, aprobación de solicitudes, activar/desactivar cuentas, resetear contraseñas.
- **Inventario** (`/admin/inventario`): Dashboard, Equipos, Préstamos y Responsables (agrupa por persona responsable o, si no tiene, por ubicación — así ningún equipo queda invisible).
- **Bitácora** (`/admin/auditoria`): registro de auditoría filtrable por usuario, área y fecha.
- **Notificaciones por correo** (opcional): aviso al administrador ante solicitud de registro o ticket nuevo, y al empleado ante cambio de estado de su ticket.

9.3 Inventario y préstamos

- Dos modos de manejo por equipo: *por unidad* (serie obligatoria, un solo préstamo a la vez) o *por cantidad* (lote sin serie individual, admite varios préstamos simultáneos según stock).
- Registro de préstamo con fecha estimada de devolución, autorización y comprobante imprimible (Word).
- Devolución total o parcial (en préstamos por cantidad).

9.4 Carga desde dispositivos móviles

1. El sistema genera un código QR único vinculado a una sesión de subida temporal.
2. El empleado escanea el QR con su celular — se abre `/mobile-upload` en el navegador del teléfono, sin necesidad de iniciar sesión.
3. Toma una foto o video (o lo selecciona de la galería) y lo sube.
4. El navegador del empleado (donde sí hay sesión) hace polling a `/mobile-upload/status/:token` y, al detectar que el archivo llegó, lo asocia automáticamente al ticket.

10. Roles y Permisos de Usuario

Rol	Nivel	Puede
Empleado	1	Portal de usuario: crear y dar seguimiento a sus propios tickets.
Técnico	2	Panel admin: gestionar tickets. Ve el listado general de usuarios solo si el superadmin le otorga el módulo "Usuarios".
Admin	3	Panel admin: gestión completa de tickets; activar/desactivar y eliminar usuarios; altas de técnicos.
Superadmin	4	Todo lo anterior + control total sin restricciones, incluyendo cambiar la contraseña de cualquier usuario.

Los módulos otorgables individualmente por el superadmin son: **Inventario**, **Préstamos**, **Bitácora**, **Solicitudes de registro** y **Usuarios** (ver detalle de columnas `acceso_*` en la sección 5.1 — tabla `usuarios`).

11. Gestión de Archivos y Subida Móvil

Los archivos multimedia NO se guardan en SQL Server. Se almacenan en disco, en:

```
C:\Users\Administrador\SistemaApp\backend\uploads\
```

En la base de datos (tabla `adjuntos`) solo se guarda la referencia — nombre original, nombre en disco, ruta, tipo MIME y tamaño.

Parámetro	Valor
Formatos soportados	JPG, JPEG, PNG, GIF, WEBP (imágenes) / MP4, MOV, AVI, WEBM, 3GP (video)
Tamaño máximo	50 MB por archivo
Nomenclatura en disco	Aleatoria (32 caracteres hex), no derivada del nombre original
Acceso URL	<code>http://10.0.1.108:3000/uploads/{'{' }nombre_archivo{'{'}}</code>
Logo de correos	<code>backend/assets/gcm.jpg</code> — copia independiente del logo del frontend, para que el envío de correos no dependa de <code>web/</code> .

12. Despliegue y Configuración del Servidor

12.1 Especificaciones del servidor

Componente	Valor
Sistema Operativo	Windows Server 2022 Datacenter Evaluation
Procesador	Intel Core i5-12450H — 2 núcleos / 2 hilos (VM)
Memoria RAM	10 GB DDR4
Disco Principal (C:)	69 GB — 50 GB disponibles
Dirección IP	10.0.1.108 (estática)
Puerto API (backend)	3000 (TCP)
Puerto Web (frontend)	3001 (TCP) — nuevo , antes todo vivía en el 3000
Puerto SQL Server	1433 (TCP)
Tipo de Red	Ethernet — Subred 10.0.1.0/23
Gateway	10.0.1.1

12.2 Procesos PM2

La aplicación ahora se gestiona como **dos procesos PM2** definidos en `ecosystem.config.js` :

App PM2	Script	Puerto	Descripción
gcm-tickets	backend/dist/server.js	3000	API. Requiere <code>npm run build</code> en <code>backend/</code> antes de <code>pm2 start</code> .
gcm-tickets-web	<code>npm run start</code> (en <code>web/</code>)	3001	Frontend Nextjs en modo producción (<code>next start</code>). Requiere <code>npm run build</code> en <code>web/</code> antes.

Ambos arrancan automáticamente al iniciar Windows mediante la tarea programada "PM2-GCM-Tickets" (`pm2 resurrect` con usuario SYSTEM), igual que antes de la migración.

12.3 Variables de entorno — `backend/.env`

Variable	Descripción
DB_SERVER / DB_PORT / DB_USER / DB_PASSWORD / DB_NAME	Conexión a SQL Server. El proceso falla al arrancar si <code>DB_PASSWORD</code> no está definida.
PORT	Puerto del backend (3000).
WEB_PORT	Puerto donde corre <code>web/</code> (3001) — usado para armar URLs en correos y QR.
WEB_APP_URL	URL pública del frontend (opcional). Si se deja vacío, se autodetecta la IP de red — necesario para que el QR de "subir desde celular" funcione desde otros equipos.
SESSION_SECRET	Clave de firma de los JWT. Mínimo 32 caracteres, el proceso no arranca si falta o es muy corta.
BCRYPT_ROUNDS	Factor de costo del hash de contraseñas (12 por defecto).

Variable	Descripción
ADMIN_PASSWORD	Contraseña inicial del usuario <code>admin</code> , solo aplica en la primera ejecución de <code>npm run setup</code> si esa cuenta no existe todavía.
SMTP_HOST / SMTP_USER / SMTP_PASS / SMTP_CC	Configuración de correo saliente (opcional).

12.4 Variables de entorno — `web/.env.local`

Variable	Descripción
NEXT_PUBLIC_API_PORT	Puerto de la API (3000). El cliente resuelve la URL completa dinámicamente a partir del hostname del navegador, para funcionar desde cualquier equipo de la LAN.
NEXT_PUBLIC_API_URL	Override opcional si se necesita apuntar a una URL fija en vez de autodetectar.

12.5 Firewall de Windows

- Puerto 3000 TCP — API (entrada)
- Puerto 3001 TCP — Frontend web (entrada) — **nuevo**
- Puerto 1433 TCP — SQL Server (entrada)

13. Respaldo y Recuperación

Respaldo automático diario vía tarea programada de Windows "GCM-Tickets-Backup-Diario".

Parámetro	Valor
Hora de ejecución	2:00 AM todos los días
Ruta de destino	C:\Backups\GCM-Tickets\{' '}YYYY-MM-DD{' '}\
Contenido	Archivo .BAK de SQL Server + .ZIP de backend\uploads\
Retención	7 días (los más antiguos se eliminan automáticamente)
Credenciales	backup.ps1 lee DB_USER / DB_PASSWORD directamente de backend\.env — ya no hay ninguna contraseña escrita en el script.

```
RESTORE DATABASE gcm_tickets FROM DISK = 'C:\Backups\GCM-Tickets\{fecha}\gcm_tickets_{fecha}.bak'  
WITH REPLACE, RECOVERY;
```

14. Mantenimiento y Comandos

Comando	Descripción
pm2 list	Ver estado de ambos procesos
pm2 logs gcm-tickets	Logs en tiempo real del backend
pm2 logs gcm-tickets-web	Logs en tiempo real del frontend
pm2 restart gcm-tickets gcm-tickets-web	Reiniciar ambos procesos
pm2 stop gcm-tickets gcm-tickets-web	Detener ambos procesos
pm2 monit	Monitor visual de CPU y memoria
pm2 save	Guardar el estado actual de los procesos

Actualizar el sistema

- Aplicar cambios al código fuente.
- Backend: `cd backend && npm install && npm run build`
- Frontend: `cd web && npm install && npm run build`
- `pm2 restart gcm-tickets gcm-tickets-web`
- Verificar logs de ambos procesos.

15. Historial de Migración

En julio de 2026 se migró el sistema completo desde un backend Express en JavaScript que también servía un frontend estático en HTML/CSS/JavaScript plano (`admin.js` , `usuario.js` , ~10,300 líneas), hacia la arquitectura actual. Objetivo: modernizar el stack sin tocar la lógica de negocio ni el diseño visual existente.

Qué cambió

- Backend Express convertido 1:1 a TypeScript, con tipos explícitos para usuarios, tickets, inventario y préstamos — misma lógica de negocio, ahora con seguridad de tipos.
- Frontend HTML/JS plano reescrito completo en Next.js (App Router) + TypeScript + Tailwind CSS, replicando visualmente las dos paletas de marca existentes (panel admin y portal de empleados).
- Reemplazo de dependencias por CDN (íconos, gráficos, Excel) por paquetes npm equivalentes: `@tabler/icons-react` , `chart.js` / `react-chartjs-2` , `xlsx` .
- Backend y frontend pasaron de un solo proceso/puerto a dos procesos independientes (3000 / 3001) con CORS y autenticación por Bearer token en vez de mismo-origen.

Qué NO cambió

- El esquema de base de datos y los stored procedures — ninguna migración de datos fue necesaria.
- La lógica de negocio de cada endpoint — se tipó, no se reescribió.
- El diseño visual — se replicó pixel a pixel el aspecto de ambos paneles, no fue un rediseño.

Auditoría de seguridad posterior

Tras completar la migración se realizó la revisión de seguridad documentada en la sección 8, que corrigió un problema de control de acceso preexistente en la gestión de tickets (no introducido por la migración), endureció la gestión de secretos, y limpió el historial de git de credenciales que habían quedado expuestas en commits anteriores a esta migración.